ROOT-CAUSE ANALYTICS · DATA ANALYTICS AND DATA SCIENCE

Debie Galleros

Data quality, root-cause analysis and AI evaluation

github.com/debiegalleros

Since May 2026 I have worked on the data behind an Amazon brand agency's platform: sales and traffic reports, advertising data, inventory, reimbursements and AI-generated product content. It runs as five apps on one shared Postgres database. I didn't build these apps. I was the only person on the team fixing their bugs: I found out why their numbers or features went wrong, submitted each fix to my manager for review, and shipped it once approved.

TIMELINE, 2026 · ● MY WORK ● PLATFORM CHANGES

Mar – May
The apps are built
Another team builds the five apps.

May
I start on bugs
I handle the first bug reports from users as the apps go into first use.

Jun
AI model retirements
Unprompted, I spot retirements on the provider's schedule and raise a fix early.

Jul
First database migration
Airtable to Supabase. Database headers stop matching the screens, and fixes pile up.

Aug
Move to Claude Code and pull requests
Code already on GitHub. I switch from Lovable's AI editor to reviewed pull requests.

Sep
Second migration starts
Still under way, so new bugs keep coming.

Nov
612 tickets closed
Across all five apps, as the only person fixing bugs.

Most cases started with a number that looked wrong. I traced it through SQL, request logs and the code that wrote it, measured how far it reached, and shipped the fix. All five apps' code lived on GitHub. Until July I made fixes through Lovable's AI editor. In August I moved to Claude Code and reviewed pull requests, which gave more reliable fixes and a documented, reviewable record of every change. Either way, I checked the AI's findings against the live database before anything went out.

612 tickets closed since May	**149** fixes merged, of 171 pull requests since Aug 6	**1,848** failed requests traced to one setting	**2,741** labelled AI review notes I tested one fix on

> "Really impressed with your deep-diving the state of the API source from Anthropic. You are using a higher level of strategic analysis now that you've been in the trenches. I am convinced that once you are able to take a breather from tactical operations, you will have insights that will allow you to make great contributions to our software strategy."
>
> My manager, Jun 20, in my second month of bug fixing, replying to my own analysis of Anthropic's model retirements, which no bug report had asked for

How it helped

What changed for the company and its clients. Business figures are left out for confidentiality.

Clients

- Profit and loss in the client portal counts credits received outside Amazon
- Brands selling in the US, Canada and Mexico see each country's own numbers
- Customer loyalty trends read one storefront at a time

Operations and analytics teams

- Agents no longer upload reports by hand or keep their own Excel sheets, which saves time and money and keeps the system up to date
- Lists and totals are complete, or show an error, in all five apps
- The cause of 37 outages in 12 days is fixed
- Wrong-account ad uploads are blocked, and 120,238 misfiled rows were kept with zero loss
- Every reimbursement claim can be found, with case actions and a claim-type filter

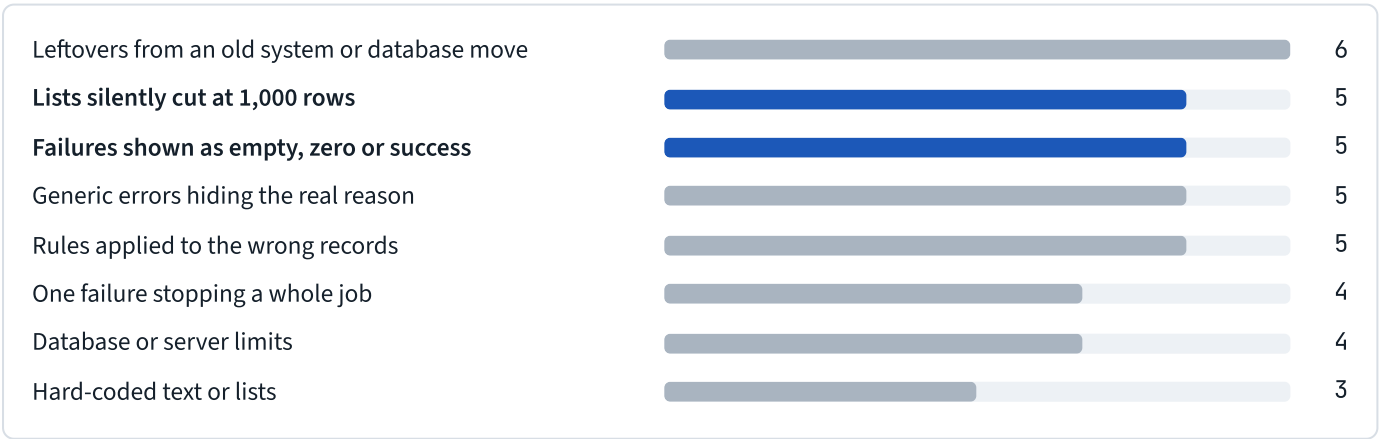
AI features

- Two June model retirements raised for a fix before the retirement date
- The next retirement forecast in June and handled three weeks early, before anything broke
- Video generation working again after Google retired its model
- The AI reviewer re-runs fails its own notes contradict; 630 had been missed
- Finished research results stay visible when the AI provider refuses calls

Management

- Bug fixing for all five apps covered by one person, with every fix reviewed before release
- A weekly fix count with its undercount found and corrected
- Daily status reports and a weekly report on every ticket

What I found most often



Root causes in the 39 tickets I wrote up from Sep 29 to Nov 11, by number of tickets. One ticket can have more than one cause. The most common one traces back to the platform's migrations. The two highlighted causes made wrong data look right: nothing on screen said a list was cut short or a request had failed.

Selected analyses

LOG ANALYSIS · ALL FIVE APPS

Outages that hit every app at once

PROBLEM — All five apps sometimes refused every request for a few seconds to five minutes, with "503 Service Unavailable" and no clear trigger.

ANALYSIS — I pulled 12 days of request logs, grouped the refused requests by minute, and lined each burst up against the database API's own log.

FINDING — 37 outages and 1,848 refused requests, 15 of the outages on a single day. Every outage minute had one failed schema reload, cut off by an 8-second limit. The slowest reload took 13.3 seconds.

OUTCOME — **Raised that one limit to 300 seconds.** Live since Nov 5. Limits on user requests stayed the same.

AI EVALUATION · CONTENT REVIEW

Testing an AI reviewer on all its past answers

PROBLEM — An AI step that reviews generated product copy failed text while its own note said the text "actually complies".

ANALYSIS — A re-run rule caught these contradictions only when the note used one of a fixed list of phrases. I measured that rule against every saved review since it started. Keyword counts swung from about 125 to 1,800 depending on the pattern, so I didn't rely on them.

FINDING — 927 saved fails said in their own note that the text passed. 630 of them were never re-run, because the phrase list missed their wording.

OUTCOME — **Replaced the phrase list with a short AI check that reads what each failing note concludes.** Tested on all 2,741 labelled review notes with the labels hidden, then verified live.

TEST ON 2,741 LABELLED NOTES	OLD PHRASE LIST	NEW AI CHECK
Contradictions caught, of 1,425	272 (19%)	1,308 (92%)
Real fails wrongly re-run, of 1,174	90 (8%)	47 (4%)

DATA RECONCILIATION · REPORT UPLOADS

Ad data filed under the wrong account

PROBLEM — Advertising bulk files for one seller account appeared under another, mixing their ad data.

ANALYSIS — I compared the account named in each of the 279 past file names with the account it was filed under. Then I matched every suspect row against its correct account on four keys.

FINDING — 5 imports contradicted their file names. Four of them had put 120,238 rows under the wrong account, and every one of those rows already existed under the right one.

OUTCOME — **Misfiled rows backed up, with zero data loss.** A file-name check now blocks wrong uploads. Run on the history, it passed all 274 correct ones and caught the 5 wrong ones.

DATA COMPLETENESS · ALL FIVE APPS

Lists that silently stopped at 1,000 rows

PROBLEM — Lists and totals came back incomplete with no error: groups missing members, report rows gone, claims missing from search.

ANALYSIS — I swept the database reads in all five apps for any that could return more than 1,000 rows. In the largest app that meant 158 server functions and 297 database function calls.

FINDING — The database API returns at most 1,000 rows per request, and the code treated that as the whole list. Reads without a fixed sort order also overlapped between pages: in one claims list, about 1 in 5 items never appeared.

OUTCOME — **Live in all five apps since Oct 21.** One shared reader returns every row or raises an error, and a CI check blocks new reads that skip it.

SEGMENTATION · MULTI-COUNTRY SELLERS

Numbers from the wrong country

PROBLEM — For brands selling in the US, Canada and Mexico, some pages counted other countries' figures as US numbers.

ANALYSIS — Before changing anything, I measured on live data which pages actually showed wrong numbers. After the change, I compared each report's output old against new, in both directions.

FINDING — The wrong pages read by account and ignored the marketplace. Some stock views quietly defaulted to the US.

OUTCOME — **Affected pages in all five apps follow the marketplace picker.** US-only reports now say so, and every other report kept exactly the same numbers.

DATA COLLECTION · GITHUB CHANGE AUDIT

A weekly count of fixes that was quietly wrong

PROBLEM — Every Monday the team needed a count of the fixes made to its server functions across all the codebases, recorded in a tracking table and reported onward.

ANALYSIS — Each week I pulled every merged pull request from GitHub, read its code changes, and recorded each verified fix with the tables and apps it touched. When the numbers looked off, I checked how they were collected.

FINDING — Two measurement errors. The audit covered four codebases when there were five, so every earlier week was undercounted. Sorting pull requests by their titles also missed real fixes, 5 in one week, because most of them mixed features and fixes.

OUTCOME — **Fixed the scope and re-swept the earlier weeks, which recovered 4 missed fixes.** Classification now reads each change, not its title. Commit authors didn't show who drove a change, so I tested another signal: it matched 6 of 6 known cases and 0 of 18 others.

AI models and pipelines

Forecast the June model retirement (Jun)	No bug report asked for this. I read Anthropic's model retirement schedule, saw that two models the apps still called would retire on Jun 15, and alerted the team and raised a fix before the retirement date. A retirement like this reaches the AI features in four of the five apps: 72 of 347 server functions today, 48 of them in the research and content app. Every hard-coded model ID and config table moved to a supported model, with clear error messages, automatic retries and an AI status badge. I also noted when the next model would retire: Sep 29.
Acted on my own forecast, three weeks early (Sep)	In early September, 10 pipeline stages and 32 files in the largest app still used the model I had flagged in June for retirement on Sep 29. I moved all four apps that used it on Sep 8, before anything broke.
Found a model name that didn't exist	The same check found a default model ID that was never released, left by a bad automated edit. No caller used it yet, so I fixed it before one did.
Video generation after Google retired its model	Google shut down its video model on Oct 22, and every video feature failed with a generic error. I moved video generation to Google's new model and made the screens show the real reason.
Research results hidden by a provider refusal	When the AI provider refused calls because the account ran out of credit, a keyword research tool marked finished analyses as failed and hid them. The results were intact. I made temporary refusals leave the work as it was.

SQL and code from my work

Claude Code wrote first drafts, and I reviewed each change before it went into a pull request. These examples show the patterns behind three of the analyses above, rewritten as short generic code. The real code belongs to the company, so none of it is copied here.

SQL · RECONCILIATION

Prove nothing is lost before removing data

Before any misfiled ad row was removed, every one had to exist under its correct account, matched on all four keys.

```
select count(*)                      as misfiled_rows,
       count(c.entity_id)            as found_under_correct_account,
       count(*) - count(c.entity_id) as would_be_lost
from misfiled_rows m
left join ad_entities c
  on  c.seller_id      = m.correct_seller_id
  and c.marketplace_id = m.marketplace_id
  and c.entity_type    = m.entity_type
  and c.entity_id      = m.entity_id;
-- Result: 120,238 misfiled, 120,238 found, 0 would be lost
```

SQL · LOG ANALYSIS AND FIX

Find the outages, then fix the setting

Refused requests grouped by minute showed each outage as a burst. Each burst matched one failed schema reload.

ANALYSIS

```
select date_trunc('minute', created_at) as minute,
       count(*)                         as refused_requests
from request_log
where status = 503
  and path like '/rest/v1/%'
group by 1
order by 1;
```

FIX

```
-- Give the schema reload room. User requests keep their own limits.
alter role authenticator set statement_timeout = '300s';
notify pgrst, 'reload config';
```

TYPESCRIPT · DATA COMPLETENESS

Read every row, or fail loudly

A plain read returns at most 1,000 rows and no error. The fix pages through in a fixed order and stops with an error instead of returning part of a list.

BEFORE

```
// Returns at most 1,000 rows. Nothing says the list was cut.
const { data } = await supabase.from("orders").select("*");
```

AFTER

```
async function fetchAll(table: string, pageSize = 1000) {
  const rows: unknown[] = [];
  for (let from = 0; ; from += pageSize) {
    const { data, error } = await supabase
      .from(table)
      .select("*")
      .order("id")                       // fixed order, so pages never overlap
      .range(from, from + pageSize - 1);
    if (error) throw error;              // fail loudly, never return part
    rows.push(...data);
    if (data.length < pageSize) return rows;
  }
}
```

More investigations

Not every investigation ended in my own pull request. In these, the lead developer shipped the fix, no code change was needed, or the fix was waiting for approval at hand-over.

1,654 failed runs in a row	An automated pipeline that groups products created nothing for three weeks. Two queued products were still filed under a former brand, and one refusal cancelled each whole run. The fix shipped and I verified it live.
Uploads reported as missing	A reconciliation sheet listed fee reports as never uploaded. I matched each gap to its cause: some imports failed without a log entry but another upload covered the same month, and the rest were older than the data available to download. No data was lost.
Wrong-size AI recommendations	An AI recommender showed wrong-size products as confident top substitutes. I found three separate causes: a candidate cap, the ranking, and an auto-approve check. The fix also clears 1,620 stale scores.
A page that failed every load	A setup step failed all 162 loads in 9 days. The page packed every template ID into the request URL, which grew to about 24,600 characters until the server refused it.
Approved copy clients never saw	Approving an older version of a product title never made it the current one, and the client portal shows only current versions. I traced 33 affected versions and designed a repair with a backup.

Other fixes I shipped, by app

Operations hub

- Reimbursement claims: case actions on every claim, a shipment column and a claim-type filter
- Claim search missed records because a list was read unsorted; the shipments tab skipped or doubled lines
- Record-link search showed "no results" for every term because the request was failing
- Account health: a clear deactivation banner, and a warning before reassigning an untriaged alert
- Compliance uploads: real error messages, several products and files per upload

Research and content

- Product type and category didn't save on the product page
- Group pickers showed every live group, and Create saved the group's products
- Sentiment records tab lists every record, with add-by-product
- Long AI content runs split into timed rounds so they stop timing out
- Review checks reworded so they apply to any brand, not one product category

Sales analytics

- Large report uploads timed out or ran out of memory; they now upload in smaller pieces
- A "confirmed no data" state so an empty week doesn't look like a missing upload
- An empty ads report is no longer refused as "wrong account"
- Weekly search reports save the week's real end date
- Upload coverage shows every active account

Client portal

- Profit and loss counts credits received outside Amazon
- Customer loyalty trends read one storefront at a time
- Content review stopped loading internal strategist notes

Team workspace

- Record numbers for every record type, annual reviews included
- Continuous-improvement tabs that showed "saved" but never saved
- Time tracking flagged finished entries as "Invalid status"

Across all apps

- Numbered every bug report and change request, in all six codebases
- Created and last-updated dates now fill in automatically

How I work

Start from the number that looks wrong	I open the user's own record and reproduce exactly what they saw.
Prove it with data	Query results, request logs and row counts. If a cause can't explain every detail in the report, I keep looking.
Measure the reach	I count every record, page and report the same cause touches before deciding on the fix.
Compare before and after	I check old against new output in both directions before calling a change safe.
Test on all the data	An AI change gets tested on every saved output, not only the example in the ticket.
Ship it as a reviewed pull request	Each fix goes on its own branch with its spec update and deploy steps, passes the tests against the latest code, and is reviewed before merge. After merge I run the deploy steps and check the result live.
Write it up	Root-cause reports, daily status reports and a weekly report to my manager. At my manager's request, I shared my reporting routine with two teammates.

Tools and skills

SQL	Postgres: joins, aggregates, time bucketing, reconciliation and data-quality checks, migrations
DATA	Amazon seller data (sales and traffic, advertising bulk files, fees, inventory, reimbursements, multiple marketplaces) and application request logs
AI AND LLMS	Evaluating model outputs against saved results, AI classification with structured output, Claude and Gemini pipelines
TOOLS	GitHub (pull requests, CI checks, reviewing AI-written changes, weekly change audits), Supabase, Claude Code, Lovable; reading TypeScript
WRITING	Root-cause reports, technical and user specs, status reports

Company, client, brand and colleague names, internal IDs and business figures are left out to keep the company's work confidential. I'm happy to walk through any of these cases in more detail.